

دورة n8n المتقدمة • محتوى تأسيسي

أساسيات n8n

كل ما يحتاج أي شخص فهمه — داخل n8n وخارجه — قبل بناء أتمتة و AI Agents
بمستوى Production

APIs • حركة البيانات • أنواع الـ AI Agents • Nodes • أنماط الإنتاج • 4 مشاريع تطبيقية

ماذا سنغطي؟

الجزء 1 ما هي الأتمتة؟ وما هو n8n؟ — المفاهيم الأساسية: Workflow، Node، Execution.

الجزء 2 أساسيات خارج n8n: كيف تتحرك البيانات، APIs، JSON، Webhooks، والمصادقة (Authentication).

الجزء 3 أنواع الـ Nodes في n8n: Triggers، Actions، Logic، Data، وكيف تتدفق الـ Items بينها.

الجزء 4 الـ AI Agents: نموذج اللغة، الأدوات (Tools)، الذاكرة (Memory)، وRAG.

الجزء 5 جاهزية الإنتاج: Error Handling، Idempotency، المراقبة، وأمان الـ Credentials.

الجزء 6 الأتمتة حسب المجال: HR، تسويق، عقارات، إرسال بريد، وScraping البيانات وتنظيفها.

الجزء 7

4 مشاريع تطبيقية صغيرة: الشرح، ال Workflow، وال Architecture لكل مشروع.

ما هي الأتمتة أصلاً؟

أي أتمتة في العالم — مهما كانت معقدة — تتكون من نفس الصيغة الثلاثية:



- **Trigger:** «وصل بريد جديد»، «الساعة 7 صباحًا»، «زائر أرسل نموذجًا».
- **Logic:** «هل البيانات صحيحة؟»، «هل هذا عميل جديد أم مكرر؟»، «أي قسم يخصّ هذا الطلب؟».
- **Action:** «أرسل بريداً»، «أضف صفًا في جدول»، «أنشئ مجلدًا»، «نبّه المدير».

الخلاصة

عندما تحلل أي مهمة تريد أتمتتها، اسأل دائمًا: ما الـ Trigger؟ ما القرارات في المنتصف؟ ما الأثر النهائي؟ — هذه العادة الذهنية أهم من أي أداة.

ما هو n8n؟

أداة أتمتة workflow automation مفتوحة المصدر تتيح ربط مئات الخدمات (Google، Slack، قواعد بيانات، أي API...) عبر واجهة رسم مرئية، مع إمكانية كتابة كود JavaScript عند الحاجة.

لماذا n8n وليس Zapier/Make؟

Self-hosting: تستضيفه على سيرفرك الخاص — بياناتك لا تغادر بنيتك التحتية، وهذا شرط أساسي لكثير من الشركات.

بدون حدود مصطنعة: لا تدفع لكل عملية تنفيذ، ولا تُقيّد بعدد خطوات.

مرونة الكود: Code node كامل + HTTP Request لأي API حتى لو لم يكن له تكامل جاهز.

المصطلحات الأساسية

Workflow: المخطط الكامل — سلسلة العقد المتصلة التي تنفذ مهمة.

Node: خطوة واحدة في ال workflow (استقبال، معالجة، إرسال...).

Execution: تشغيل واحدة فعلية لل workflow — لها سجل كامل يمكن فتحه ومعاينة بيانات كل عقدة فيها.

Canvas: مساحة الرسم التي تبني فيها ال workflow بالسحب والإفلات.

تشرح ال Workflow: كيف تتحرك البيانات بين العقد؟

كل عقدة تستقبل قائمة من العناصر (**Items**)، تعالجها، ثم تُخرج قائمة items جديدة للعقدة التالية. كل item هو كائن JSON واحد.

ما يخرج من عقدة «قراءة جدول الموظفين»

```
[ { "name": "Sara", "dept": "Design" }, { "name": "Omar", "dept": "IT" }, { "name": "Lina", "dept": "HR" } ]
```

العقدة التالية «إرسال بريد» تعمل 3 مرات

```
Item 1 → email to Sara Item 2 → email to Omar Item 3 → email to Lina
```

القاعدة الذهبية

أغلب العقد تنفّذ **مرة لكل item** تلقائيًا — لا تحتاج حلقة تكرار يدوية. إذا وصلها 100 عنصر، سترسل 100 بريدًا. هذا الفهم يمنع أكثر الأخطاء شيوعًا عند المبتدئين.

كيف تتحرك البيانات على الإنترنت؟

قبل أي أتمتة، يجب فهم النموذج الأساسي للاتصال: **Request / Response**.



- ال Client **يسأل دائماً أولاً** — السيرفر لا يبادر بالإرسال (إلا في حالة ال Webhooks، وسنصل لها).
- كل طلب يذهب إلى **عنوان URL** محدد، وكل استجابة تعود مع **Status Code** يلخص النتيجة.
- البيانات المتبادلة بين الأنظمة اليوم تكون غالباً بصيغة **JSON**.

ما هو ال API؟

ال **API (Application Programming Interface)** هو «باب رسمي» تفتحه خدمة ما ليتعامل معها الكود بدل البشر. الموقع واجهة للإنسان، وال API واجهة للبرامج — نفس البيانات، بابان مختلفان.

HTTP Method	المعنى	مثال واقعي
GET	اقرأ بيانات — بدون أي تعديل	جلب حالة الطقس، قراءة قائمة العملاء
POST	أنشئ شيئًا جديدًا	إضافة عميل جديد، إرسال رسالة
PUT / PATCH	عدّل شيئًا موجودًا	تحديث بريد موظف، تغيير حالة طلب
DELETE	احذف	حذف سجل، إلغاء حجز

لماذا يهم في n8n؟

كل عقدة تكامل في n8n (Gmail، Sheets، Slack...) هي مجرد غلاف جميل حول API. وعندما لا يوجد تكامل جاهز لخدمة ما، عقدة **HTTP Request** تتيح مخاطبة أي API في العالم مباشرة — لذلك من يفهم ال APIs لا يقف عند حدود قائمة التكاملات أبدًا.

JSON — لغة البيانات التي يتكلمها كل شيء

JSON (JavaScript Object Notation) هو تنسيق نصي بسيط لتمثيل البيانات. كل ما يمر عبر n8n — مدخلات، مخرجات، ردود APIs — هو JSON.

البنية الأساسية

```
{ "name": "Sara", // نص (string) "age": 28, // رقم (number) "active": true, // منطقي (boolean)
  "skills": ["n8n", "SQL"] // مصفوفة (array) }
```

التداخل (nesting) + الوصول للقيم

```
{ "employee": { "name": "Sara", "manager": { "email": "ali@corp.com" } } } // للوصول لبريد المدير:
{{ $json.employee.manager.email }}
```

اقرأ أي JSON من الخارج للداخل: كائن ← داخله حقول ← بعض الحقول كائنات أو مصفوفات بدورها. القدرة على النظر إلى استجابة API ضخمة واستخراج «مسار» القيمة المطلوبة هي المهارة الأكثر استخدامًا يوميًا في n8n.

Status Codes والمصادقة (Authentication)

أكواد الحالة التي ستقابلها يوميًا

الكود	المعنى	ماذا تفعل؟
200 / 201	نجاح / تم الإنشاء	أكمل طبيعيًا
400	طلبك أنت خاطئ (بيانات ناقصة/غير صالحة)	صحّح البيانات — الإعادة لن تنفع
401 / 403	مشكلة مصادقة / صلاحيات	افحص الـ credential — لا Retry
429	تجاوزت حد الطلبات (rate limit)	Retry مع انتظار أطول
500 / 503	السيرفر نفسه متعطل مؤقتًا	Retry with backoff

طرق المصادقة الشائعة

- **API Key**: مفتاح نصي ثابت يُرسل مع كل طلب (في header أو query) — الأبسط والأكثر شيوعًا.
- **Bearer Token**: رمز مؤقت في header بصيغة `Authorization: Bearer <token>`.

- **OAuth2:** المستخدم يوافق عبر شاشة رسمية (مثل «Sign in with Google») والنظام يحصل على token محدود الصلاحيات قابل للتجديد — هذا ما تستخدمه عقد Google في n8n.

Webhook مقابل Polling — كيف نعرف أن شيئًا حدث؟

Polling — «اسأل كل فترة»

n8n يسأل الخدمة بشكل دوري: «هل من جديد؟» عبر Schedule Trigger + طلب GET.

مثال

كل 15 دقيقة: افحص إن كان هناك صف جديد في الجدول.

العيب: تأخير حتى الفحص التالي + طلبات كثيرة بلا فائدة أغلب الوقت.

Webhook — «أخبرني فور الحدث»

أنت تعطي الخدمة رابط URL خاص بك (يولده Webhook node)، وهي تستدعيه فورًا عند وقوع الحدث.

مثال

نموذج تسويقي يُرسل بيانات كل Lead جديد للحظة تسجيله.

الميزة: فوري وفعال — وهو أساس أغلب أنظمة الإنتاج.

تنبيه Production مهم

الخدمات المرسلَة تُعيد إرسال ال Webhook عند أي تأخر في الرد (retry behavior) — لذلك أي workflow يبدأ ب Webhook يجب أن يتحمل استقبال **نفس الطلب مرتين**. هذا بالضبط سبب حاجتنا لمفهوم Idempotency القادم في الجزء الخامس.

خريطة أنواع العقد في n8n

مئات العقد في n8n تنتمي كلها إلى 5 عائلات فقط. من يفهم العائلات يستطيع بناء أي شيء:

Triggers

تبدأ التنفيذ: Form، Webhook، Schedule، Manual، و Triggers التطبيقات (بريد جديد، صف جديد...).

Actions

تنفذ أثرًا في خدمة خارجية: HTTP Request، Slack، Google Sheets، Gmail...

Logic

تتحكم بمسار التنفيذ: Loop، Wait، Merge، Filter، Switch، IF.

Data

تعيد تشكيل البيانات نفسها: Remove Duplicates، Split Out، Aggregate، Code، Set/Edit Fields.

AI

عقد الذكاء الاصطناعي: Memory، Vector Stores، Embeddings، LLM Chain، AI Agent.

عائلة ال Triggers — من أين يبدأ كل شيء

العقدة	متى تستخدمها؟	مثال
Manual Trigger	أثناء التطوير والاختبار فقط	تجربة ال workflow قبل التفعيل
Schedule Trigger	مهام دورية بوقت معلوم	تقرير يومي 7 صباحًا، فحص كل ساعة
Webhook	استقبال فوري من نظام خارجي	Lead جديد من نموذج إعلاني، حدث من نظام ATS
Form Trigger	إدخال بشري عبر واجهة يولدها n8n نفسه	فريق HR يُدخل بيانات موظف جديد
App Triggers	حدث داخل تطبيق مربوط	Gmail Trigger: وصل بريد جديد بمعايير محددة

قاعدة اختيار سريعة

إدخال بشري → Form. نظام يخاطب نظامًا → Webhook. وقت معلوم → Schedule. حدث داخل تطبيق مدعوم → App Trigger الجاهز أو فر جهدًا من webhook يدوي.

عائلة ال Actions — حيث يقع الأثر الفعلي

عقد التكاملات الجاهزة تغطي مئات الخدمات، وكل واحدة تعرض **Operations** متعددة (أرسل، اقرأ، حدّث، احذف...):

- **Gmail / Send Email**: إرسال بريد — العقدة التي ستستخدمها كل مشاريعنا الأربعة.
- **Google Sheets: Append / Get / Update rows** — أبسط «قاعدة بيانات» للبدء وأداة التتبع الأشهر.
- **Google Drive / Docs / Calendar**: ملفات ومستندات وأحداث — عماد نظام HR Onboarding.
- **Slack / Telegram / WhatsApp**: إشعارات فورية للفرق.

العقدة الأهم على الإطلاق: **HTTP Request**

عندما لا تجد تكاملًا جاهزًا لخدمة ما، عقدة **HTTP Request** تخاطب أي API مباشرة: تحدد ال URL وال Method وال Headers وال Body بنفسك. إتقانها = لا حدود لما يمكن ربطه. وهي أيضًا بوابة ال Scraping التي سنراها في الجزء السادس.

عائلة ال Logic — التحكم بمسار التنفيذ

العقدة	ماذا تفعل؟	مثال
IF	تفرّع ثنائي: true / false	هل البريد صالح؟ نعم → أكمل، لا → ارفض برسالة
Switch	تفرّع متعدد المسارات	وجّه الطلب حسب القسم: HR / IT / Finance
Filter	يُبقى items تحقق شرطاً ويُسقط الباقي	أبقى فقط العقارات تحت 100 ألف
Merge	يجمع مخرجات فرعين في مسار واحد	دمج نتائج Doc وال Calendar بعد تنفيذهما بالتوازي
Wait	يُجمّد التنفيذ حتى وقت أو حدث	انتظار موافقة المدير (Approval Gate)
Loop Over Items	معالجة على دفعات محكومة	معالجة 500 سجل بدفعات من 50 لتفادي rate limits

الفرق بين IF و Filter — سؤال شائع

IF يقسم المسار لفرعين وكلاهما قد يُكمل بمنطق مختلف. Filter لا يتفرع — فقط يُسقط العناصر غير المطابقة ويكمل بمسار واحد.

عائلة ال Data — إعادة تشكيل البيانات نفسها

العقدة	ماذا تفعل؟	مثال
Set / Edit Fields	إنشاء أو تعديل حقول بدون كود	تجهيز نص رسالة البريد من عدة حقول
Code	JavaScript كامل عندما يعجز ال no-code	مقارنة قائمتين، تنظيف نصوص معقد
Split Out	يفكك مصفوفة داخل item واحد إلى items منفصلة	استجابة API فيها 20 مقالاً داخل حقل واحد → 20 item
Aggregate	العكس: يجمع عدة items في item واحد	دمج 5 مقالات جديدة في نشرة بريدية واحدة
Remove Duplicates	إسقاط التكرارات حسب حقل معين	إزالة عناوين URL المكررة بعد scraping
Sort / Limit	ترتيب وتحديد عدد	أحدث 10 عقارات فقط في التقرير

ثنائية تتكرر في كل مشروع

Split Out ثم Aggregate: فكّك البيانات لتعالج كل عنصر على حدة، ثم أعد تجميعها لإرسال نتيجة واحدة. ستستخدم هذا النمط في مشروع نشرة الـ RSS حرفيًا.